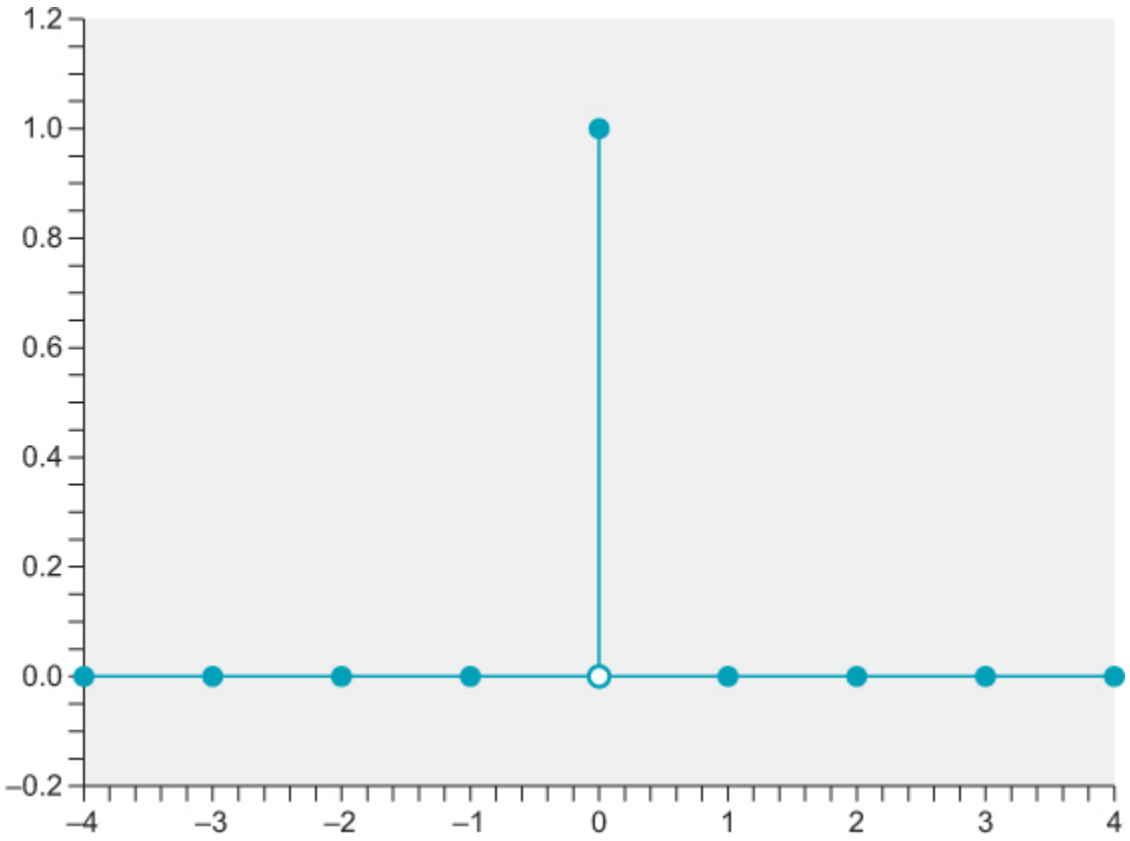


Let’s first start with how we’re going to represent and work with value distributions. As we did in the section on probability theory, we will represent a discrete probability distribution over rewards using two numpy arrays. One numpy array will be the possible outcomes (i.e., the *support* of the distribution), and the other will be an equal-sized array storing the probabilities for each associated outcome. Recall, if we take the inner product between the support array and the probability array, we get the expected reward of the distribution.

One problem with the way we’re representing the value distribution, $Z(s,a)$, is that since our array is a finite size, we can only represent a finite number of outcomes. In some cases, the rewards are usually restricted within some fixed, finite range, but in the stock market, for example, the amount of money you can make or lose is theoretically unlimited. With our method, we have to choose a minimum and maximum value that we can represent. This limitation has been solved in a follow-up paper by Dabney et al., “Distributional Reinforcement Learning with Quantile Regression” (2017). We will briefly discuss their approach at the end of the chapter.